

THE UNIVERSITY OF CHICAGO



THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

# Lecture 4: AI Agents & Tool Use

Building an Application Routing Agent

University of Chicago

# Outline

1. Where We Are and Where We're Going
2. AI Agents
3. Today’s exercise: Agentic resume review assistant

# Where We Are and Where We're Going

# The Story So Far

**Lecture 1**: Foundations

- LLMs, tokens, context windows, APIs, pricing

**Lecture 2**: Building a System

- Vertical slices and Crawl-Walk-Run
- Built a resume scorer: one prompt → one score

**Lecture 3**: Making It Good

- Decomposition, citations, few-shot examples, temperature
- Improved the scorer until gold and silver resumes separate cleanly

**Result**: We have a reliable process for resume → score.
**Question**: How do we use this to enable a better workflow?

# What We Want to Build

**Ultimately, a human will be making hiring decisions.** The AI system needs to effectively assist the human workflow.

For this we need:

- **An interface** through which a human user can interact with the system. Our interface will be..
  - **A chat** that the user can talk to in ordinary language and…
  - **A directory** where the user can put resumes to be reviewed.
- **An agentic AI** that can take the user's instructions from the chat and use them to call tools to interact with the available resumes instead of relying on data already in the context window.

# AI Agents

# What is an AI Agent?

An **AI Agent** is an LLM that dynamically uses **tools** to interact with the outside world.

Three components:

- **A prompt** with a tool registry that tells an LLM how to “use” the tools.
  - The only thing LLMs can do is output tokens; so when we say an LLM “used a tool,” we mean it outputted a series of tokens that are understood to mean “use the tool”
- **An LLM** that “uses” the tools, per the instructions in the system prompt, and inserts relevant results back into the LLM context.
- **A harness** that watches the LLM’s token outputs and executes tool use when in response to the specified LLM outputs.

All of this can be short-handed as the agent “doing things”

# What can agents do?

While a basic LLM can only generate text based on its training data, an agent can interact with “tools" like…

- **File System Tools**: Read/write files, list directories
- **Database Tools**: Query databases, insert/update/delete records
- **API Tools**: Call REST APIs, interact with web services
- **Code Execution Tools**: Run Python code, execute shell commands
- **Web Tools**: Search the web, scrape websites
- ...and more

# The Agent Loop

We can simplify this into “The Agent Loop"

Observe → Think → Act → Observe → …

1. **Observe**: Add input (user query, tool results, etc) to context
2. **Think**: Process information and decide what to do next
3. **Act**: Execute actions (call tools, respond to user, etc)

# The Tools Registry

**What makes this all possible is a tools registry, which tells the LLM how to “use” a tool and tells the harness how to know that the LLM is “using” a tool.**

**A registry has:**

- A **name**, which the LLM will use to call this tool
- A **description** of the tool, for the LLM to read
- A **schema**, which defines what information the LLM should provide, beyond the name
- A **function** that the harness will run when the tool is used

# The Tools Registry

**What makes this all possible is a tools registry, which tells the LLM how to “use” a tool and tells the harness how to know that the LLM is “using” a tool.**

**Each tool in a registry has:**

- A **name**, which the LLM will use to call this tool
- A **description** of the tool, for the LLM to read
- A **schema**, which defines what information the LLM should provide, beyond the name
- A **function** that the harness will run when the tool is used

In our example, the **function** will be a Python function called and run within the Python harness.

In many systems, the function is instead a stub that forwards the request to a separate program, usually over a standard protocol like Model Context Protocol (MCP).

# Example: Weather Query (Context)

**Agent has access to a web search tool**

**User asks**: “What's the weather tomorrow in Chicago?"

**Context window contains**:

- **System Prompt:**
  - `Use the tools below to answer the user's query. Return your decision as JSON: {"tool": ..., "parameters": {...}, "reasoning": "..."}"`
- **Tool Registry**:
  - `web_search(query: str) -> str`: Search the web and return results
  - `done(message: str)`: Signal completion with a final message
- **User Query**: “What's the weather tomorrow in Chicago?"

**CODE sends this to LLM**

# Example: Weather Query (LLM Response)

**LLM Response (Turn 1)**:

**JSON Output**

```
{
  "tool": "web_search",
  "parameters": {
    "query": "Chicago weather tomorrow forecast"
  },
  "reasoning": "Need to search for current weather
               forecast to answer the user's question"
}
```

# Example: Weather Query (Harness executes search)

**Harness (Turn 1):**

1. Parses this JSON
2. Executes web_search("Chicago weather tomorrow forecast")
3. Gets result: “Tomorrow: 45 deg F, partly cloudy, 20% chance of rain"
4. Adds result to context for next turn

# Example: Weather Query (Turn 2)

**Updated context window now includes**:

- Original query
- Tool registry
- Previous action: Called `web_search`
- Previous result: “Tomorrow: 45 deg F, partly cloudy, 20% chance of rain"

# Example: Weather Query (Turn 2, LLM Response)

**LLM Response (Turn 2)**:

**JSON Output**

```
{
  "tool": "done",
  "parameters": {
    "message": "Tomorrow in Chicago will be 45 deg F with
               partly cloudy skies and a 20% chance of rain."
  },
  "reasoning": "I have the weather information needed
               to answer the user's question"
}
```

# Example: Weather Query (Turn 2, Harness responds)

**Harness**

- Since the tool is “done", the code knows to stop the loop, send the response back to the user, and wait for a response

# Implementing the Loop in Code

## This is just a simple loop! Pseudo-code:

```
action_history = []
for turn in range(1, max_turns):
    decision = call_llm(context, tools, action_history)     # LLM

    tool_name = decision['tool']
    params = decision['parameters']
    result = execute_tool(tool_name, params) # execute tool

    action_history.append({ 'tool': tool_name, 'result': result}) # update context

    if tool_name == 'done':     # Check if done
        break
```

# Real-World and Ethical Considerations

**Agents act in the world — that raises the stakes.**

**Engineering**

- **Safety**: Max turn limits, cost caps, emergency stops
- **Observability**: Log every tool call, track costs, monitor success rates
- **Reliability**: Validate parameters, retry on transient errors, fall back to humans

**Ethics**

- **Bias**: Agents perpetuate training data biases — resume screening can disadvantage demographics
- **Transparency**: Decisions must be explainable to stakeholders
- **Accountability**: Who is responsible when the agent is wrong?

**The common answer: human-in-the-loop**

- Design explicit “flag for review” tools for high-stakes, edge-case, or low-confidence decisions
- Keep an audit trail of every action and why it was taken

# Today’s exercise:
# Agentic resume review assistant

# What we want to build

**Ultimately, a human will be making hiring decisions.** The AI system needs to effectively assist the human workflow.

Our agent will be simple, it will ultimately only do one thing:

**Return a list of annotated resumes as requested by the user.**

*Note: Our agent will never “talk” to the user. Many agents talk, but they don’t have to.*

# The underlying database

| resume_id | score | score_reasoning |
|---|---|---|
| 10089434 | 37 | Systems administrator/IT technician focused on... |
| 10247517 | 33 | IT manager with long background in network eng... |
| 10265057 | 34 | RF systems/hardware engineer working on RF and... |
| 10553553 | 34 | IT manager running a small local IT team (Acti... |
| 10641230 | 38 | IT management/computer repair professional wit... |
| ... | ... | ... |
| 95011061 | 9 | Healthcare field manager and ACA navigator wit... |
| g01 | 95 | Senior full-stack engineer with 8 years of C#/... |
| g02 | 97 | 10 years building enterprise apps with C#/.NET... |
| s01 | 71 | Junior developer with only 1.5 years of experi... |
| s02 | 73 | Senior front-end specialist with 10 years of J... |

134 rows × 2 columns

# Tools you'll be working with: Respond

Query the resume database and give the results to the user.

# Tools you'll be working with: Respond

**User:** Which candidates scored 90 or above?

**LLM:**
```
{
    "tool": "respond",
    "properties": {
        "select": ["score", "score_reasoning"],
        "where": "score >= 90"
    }
}
```

**Harness:** Fetches from database per provided `select` and `where` from the LLM and returns to the user.

# Tools you'll be working with: Augment

Make individual LLM calls to read the resumes and add/update a column in the database with new, relevant information.

# Tools you'll be working with: Augment

**User:** Which users have experience with Python?

**LLM:**
```
{
    "tool": "augment",
    "properties": {
        "column": "python_experience",
        "prompt": "Does this person have professional experience with Python?",
        "type": "string"
    }
}
```

**Harness:** Generates 138 LLM calls to fill out this column for each resume, and then adds the column to the database

# Today's Workflow

**You will build a resume review assistant**

Open the notebook:

`Lecture_4_hr_query_agent.ipynb`

Run the existing code and complete the exercises:

**Stage 1:** Respond-only; **Exercise:** Add sorting

**Stage 2:** Augment; **Exercise:** Selective augmentation

**Bonus exercises:** Reasoning, delete tool, or choose your own adventure and add your own tool!